



Miner Challenges

Auteurs : DELISLE Baptiste & DEVINCK Thomas

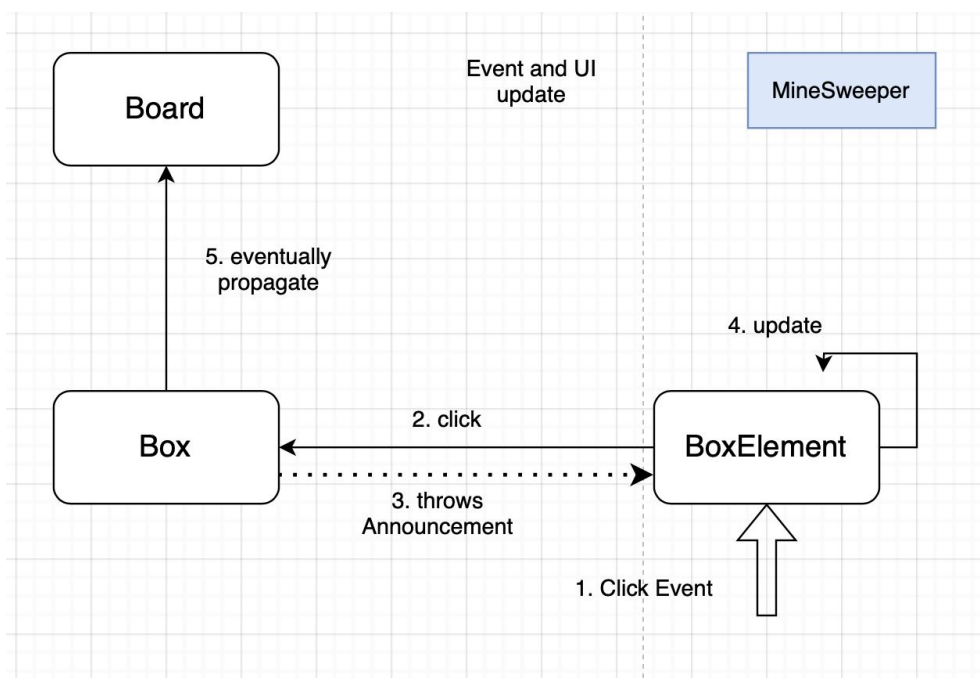
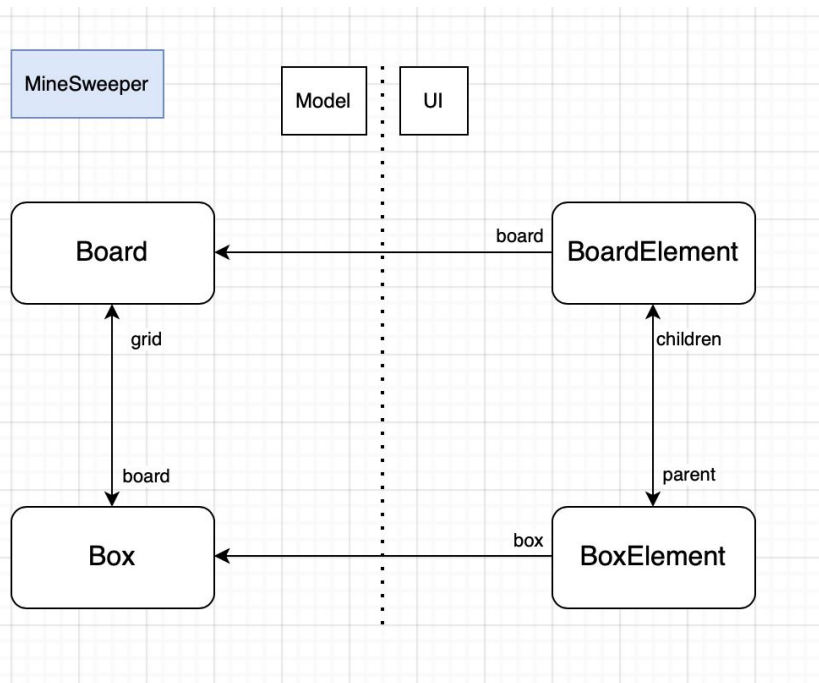
Code : <https://github.com/thomasdvck/Myg>

Contexte



Démineur

Design de base



CHALLENGE N°1 : LE COMPTEUR DE COUPS

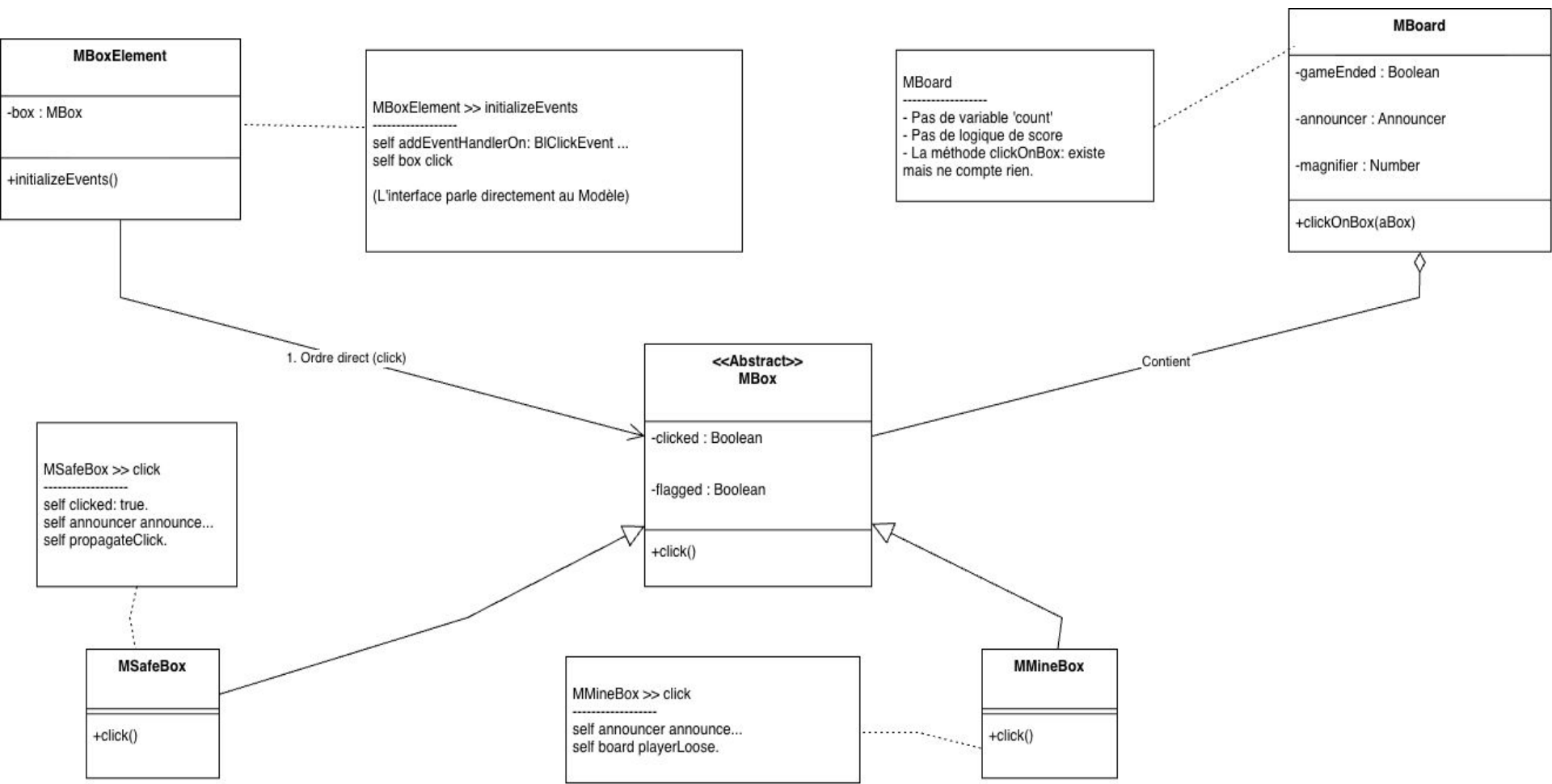
- Objectif : Afficher le nombre de coups joués en temps réel.



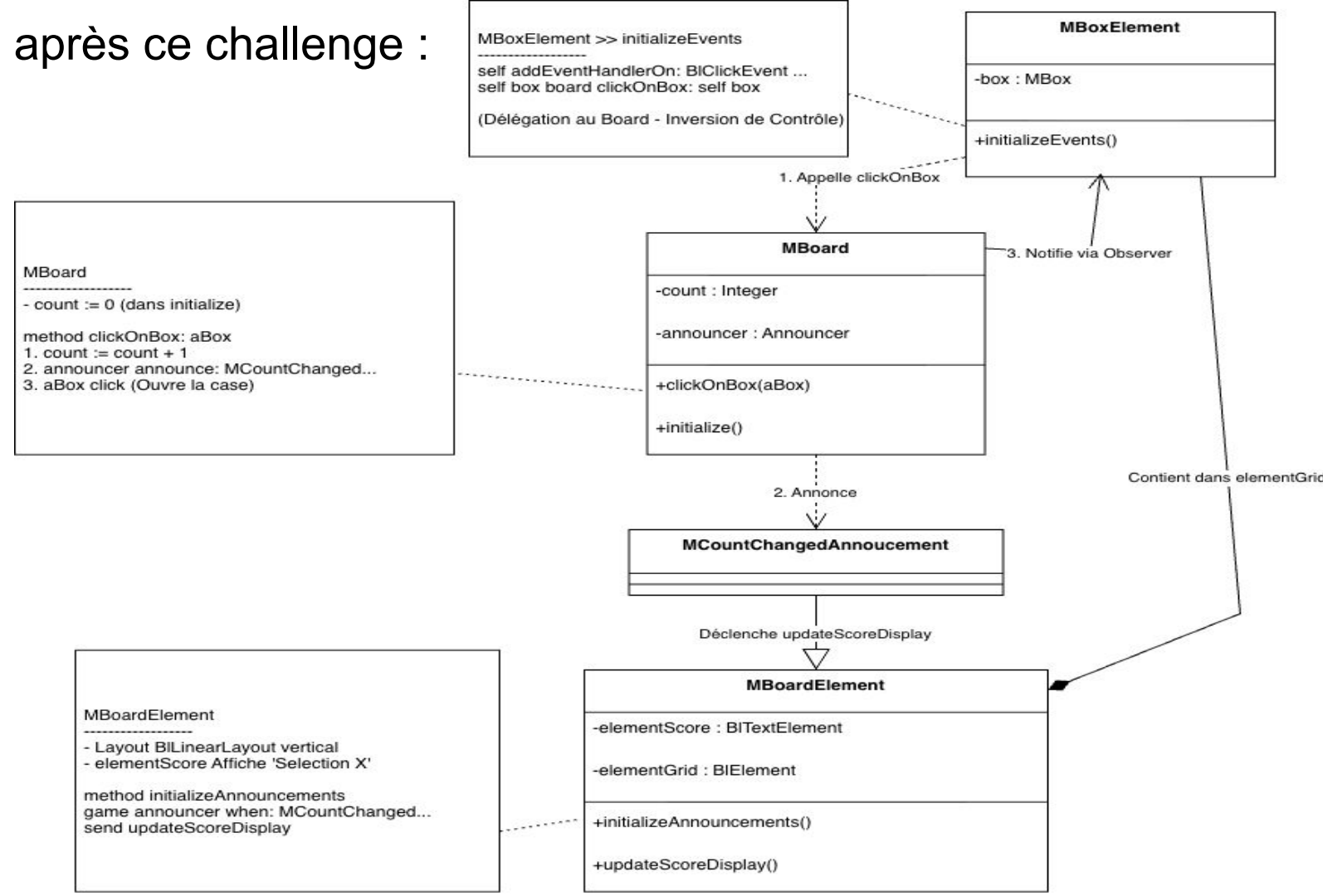
COUPS : 4

⚠ Distinguer l'**Intention** (Clic Joueur) de la **Conséquence** (Cases Ouvertes).

Design avant ce challenge :

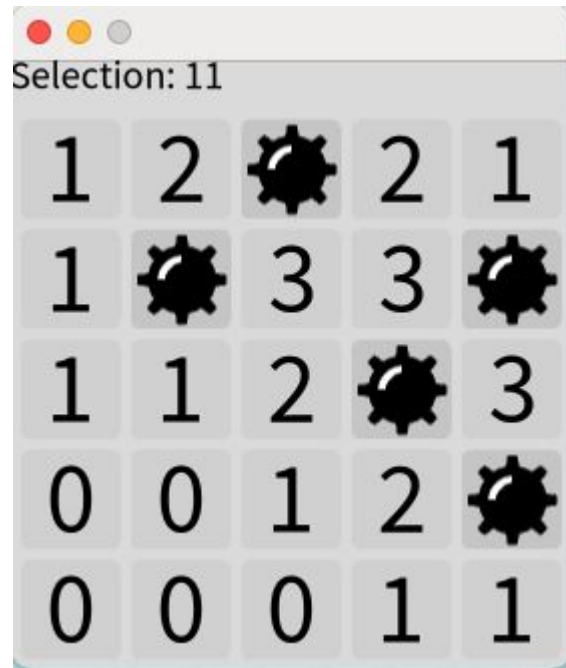


Design après ce challenge :



Éléments Importants & résultat

- Design Pattern Observer maintenue et élargit.
- Refactoring Graphique : Passage ou Layout Vertical
 1. **Problème** : MBoardElement => simple BGridLayout(Grille)
 2. **Blocage** : impossible d'ajouter du texte sans casser la grille
 3. **Solution** : Transformer le conteneur en BLinearLayout vertical
 - i. Structure :
 1. elementScore (Texte) en haut
 2. elementGrid (Grille) en dessous



CHALLENGE N°2 : POSSIBILITÉ DE CHANGER LA TAILLE DU PLATEAU

SMALL (5x5)

🌀	2	🌀	🌀	2	1	2	🌀	2	1
1	2	2	3	3	🌀	2	2	3	🌀
0	0	0	1	🌀	3	2	1	🌀	2
0	0	0	1	2	🌀	1	1	2	2
0	0	0	0	1	1	1	1	2	🌀
1	1	0	0	0	0	0	1	🌀	2
2	🌀	0	0	0	0	1	2	3	2
🌀	3	1	1	1	1	2	🌀	2	🌀
🌀	2	1	🌀	2	3	🌀	3	2	1
1	1	1	2	🌀	3	🌀	2	0	0

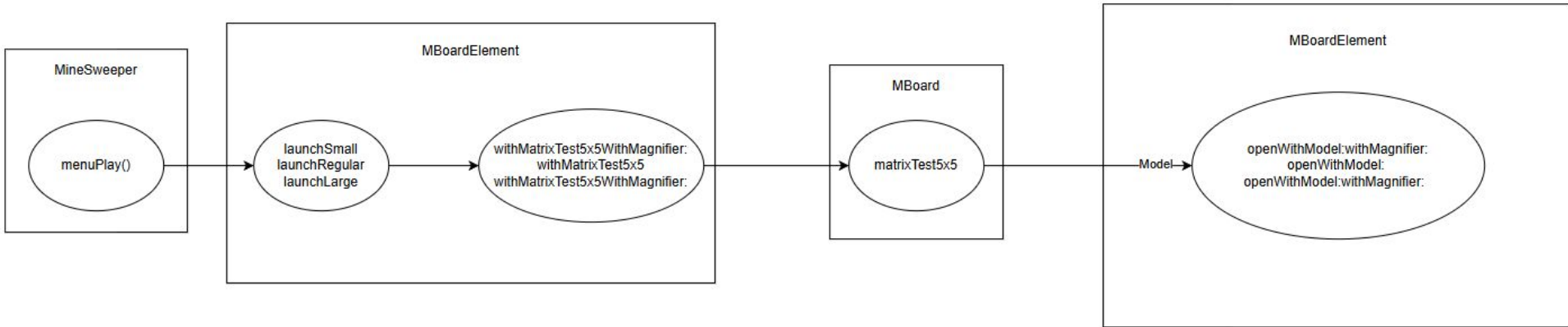
MEDIUM (10x10)

🌀	1	🌀	0	0	2	🌀	3	🌀	1
2	2	0	1	1	3	🌀	3	1	🌀
🌀	2	0	2	🌀	4	2	2	0	2
🌀	4	1	3	🌀	3	🌀	1	0	2
🌀	4	🌀	2	1	2	1	1	0	🌀
🌀	3	1	1	0	0	0	1	1	2
2	2	0	1	1	1	0	2	🌀	2
🌀	3	1	2	🌀	2	1	3	🌀	2
1	2	🌀	2	1	3	🌀	4	2	1
0	1	1	1	0	2	🌀	3	🌀	1

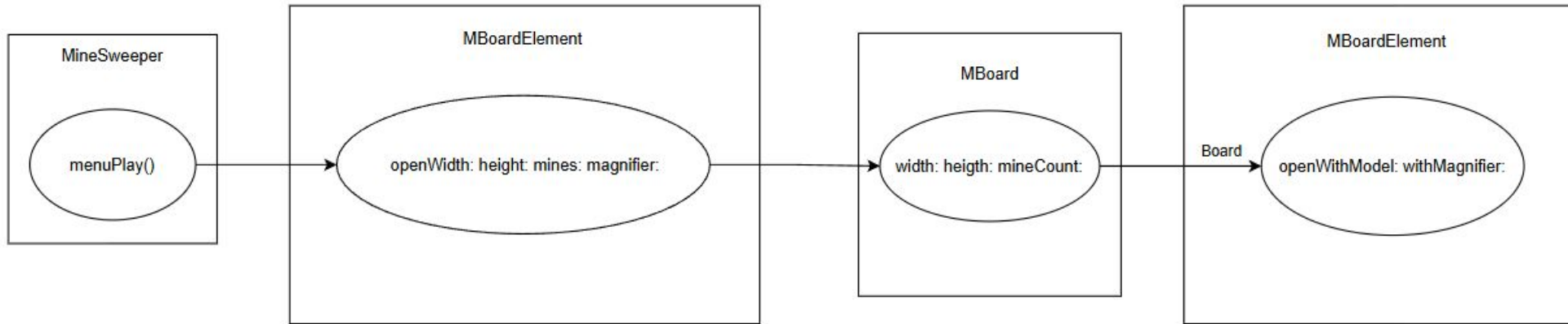
LARGE (20x20)

		2	0	2	1	2	🌀	2	2	1	2	3	🌀	2	2	1	2	2	1
1	🌀	1	3	2	2	3	2	2	1	1	3	🌀	2	1	1	2	3	🌀	2
2	2	3	3	3	0	🌀	2	1	2	2	2	3	1	0	1	1	3	🌀	2
0	0	0	0	0	0	3	🌀	2	3	🌀	2	3	0	1	1	2	2	2	2
0	0	0	1	0	1	1	3	🌀	3	2	1	1	1	1	🌀	2	2	1	2
0	🌀	1	0	0	0	1	2	0	0	0	0	1	1	2	2	2	1	1	2
0	0	0	0	1	0	1	2	🌀	3	1	1	1	🌀	2	1	1	1	🌀	1
0	1	0	0	1	1	1	2	2	🌀	2	1	2	2	3	1	2	1	1	2
0	1	1	1	0	0	0	1	2	0	2	3	3	3	2	3	1	0	0	0
0	2	🌀	1	1	1	0	1	1	1	🌀	3	🌀	1	2	1	2	🌀	1	1
0	3	1	0	1	1	1	0	1	2	2	2	2	3	2	1	2	2	4	2
2	2	1	1	1	🌀	2	1	1	0	0	1	1	2	2	🌀	3	2	3	3
0	2	1	1	1	1	🌀	1	1	0	0	0	1	1	2	2	3	2	🌀	1
2	2	1	1	1	1	1	1	1	1	1	1	1	🌀	2	2	2	3	0	2
1	1	1	🌀	1	1	0	1	1	0	1	2	3	3	2	🌀	2	1	1	1
1	🌀	2	2	1	1	0	0	1	2	🌀	2	3	4	5	2	2	1	1	0
2	2	3	2	🌀	1	1	1	1	2	2	3	4	3	5	0	2	1	0	1
1	3	🌀	3	2	2	2	2	🌀	4	3	2	3	🌀	3	1	2	3	4	🌀
🌀	3	1	1	1	🌀	2	2	2	🌀	3	2	2	3	3	🌀	2	3	2	1
1	0	0	0	2	2	2	1	1	1	3	2	0	1	0	0	🌀	0	0	0

Design avant ce challenge :



Design après ce challenge :



CHALLENGE N°3 : INTRODUCTION DE DIFFÉRENTS ALGORITHMES DE PLACEMENT DE MINES

ALGORITHME 1 : PARTIE
HAUTE DE LA GRILLE SAFE

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
1	2	2	1	0	1	1	1	0	0
1	⚡	⚡	1	0	2	⚡	3	1	1
3	4	4	3	2	3	⚡	4	⚡	2
⚡	⚡	4	⚡	⚡	4	3	⚡	4	⚡
4	⚡	5	⚡	⚡	5	⚡	4	4	⚡
2	⚡	3	2	3	⚡	⚡	3	⚡	2

ALGORITHME 2 : JAMAIS SUR
COLONNE PAIRE

⚡	1	⚡	0	0	2	⚡	3	⚡	1
2	2	0	1	1	3	⚡	3	1	⚡
⚡	2	0	2	⚡	4	2	2	0	2
⚡	4	1	3	⚡	3	⚡	1	0	2
⚡	4	⚡	2	1	2	1	1	0	⚡
⚡	3	1	1	0	0	0	1	1	2
2	2	0	1	1	1	0	2	⚡	2
⚡	3	1	2	⚡	2	1	3	⚡	2
1	2	⚡	2	1	3	⚡	4	2	1
0	1	1	1	0	2	⚡	3	⚡	1

ALGORITHME 3 : ALÉATOIRE
(RANDOM)

⚡	2	⚡	⚡	2	1	2	⚡	2	1
1	2	2	3	3	⚡	2	2	3	⚡
0	0	0	1	⚡	3	2	1	⚡	2
0	0	0	1	2	⚡	1	1	2	2
0	0	0	0	1	1	1	1	2	⚡
1	1	0	0	0	0	0	1	⚡	2
⚡	2	0	0	0	0	1	2	3	2
⚡	3	1	1	1	1	2	⚡	2	⚡
⚡	2	1	⚡	2	3	⚡	3	2	1
1	1	1	2	⚡	3	⚡	2	0	0

Design avant ce challenge :

MBoard class >> width:w height:h mineCount:m

[... Logique Aléatoire En Dur ...]

1 to: m do: [
 x := (1 to: w) atRandom.
 y := (1 to: h) atRandom.
 ...
]

--> Algorithme figé. Impossible de changer la règle sans modifier cette méthode.

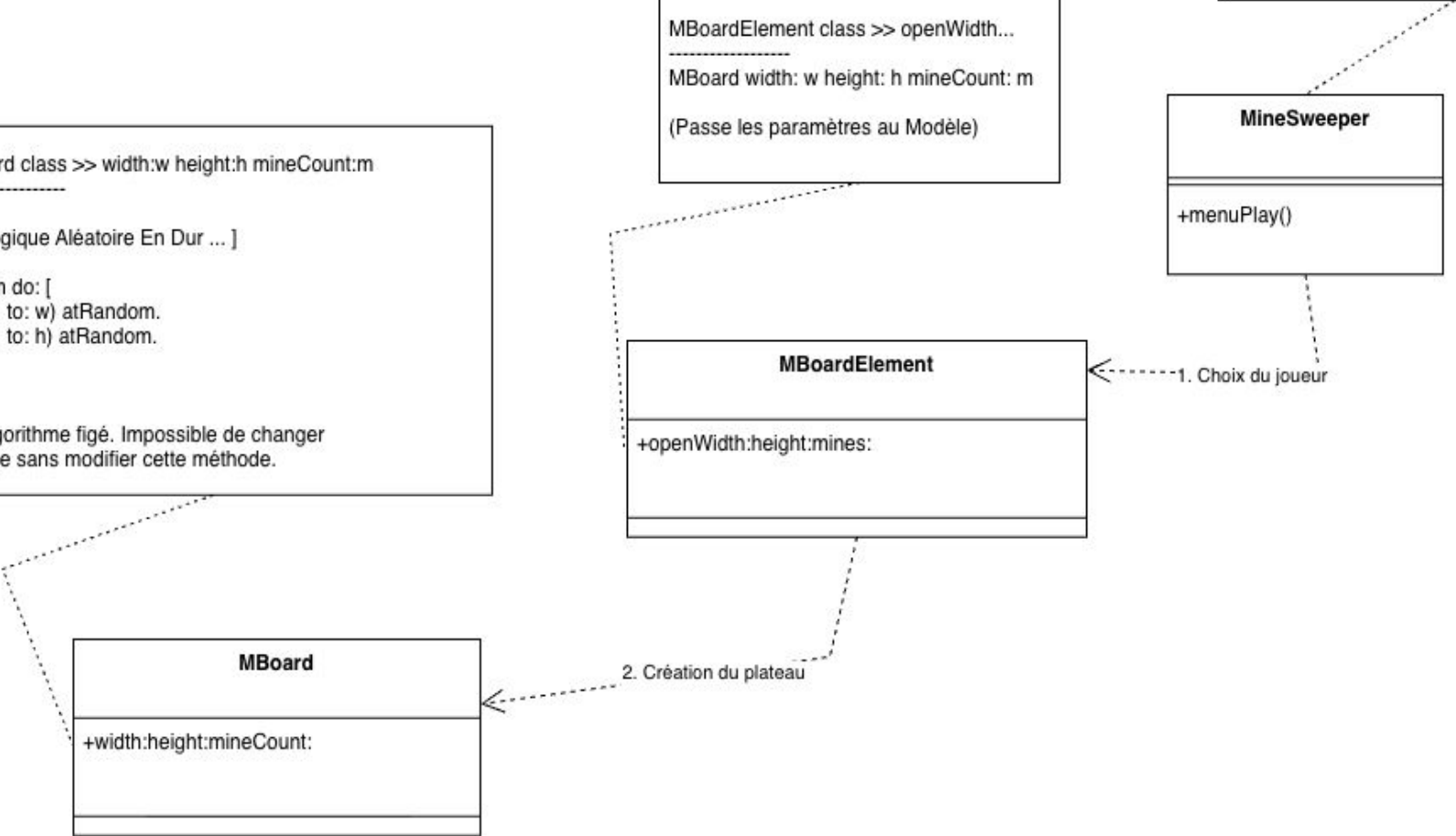
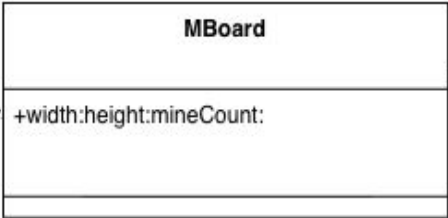
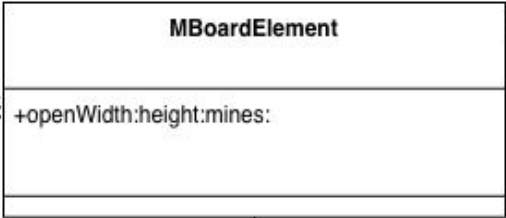
MBoardElement class >> openWidth...

MBoard width: w height: h mineCount: m

(Passe les paramètres au Modèle)

MineSweeper (Menu)

Propose Small, Medium, Large.
Envoie (width, height, mines)
à l'interface.



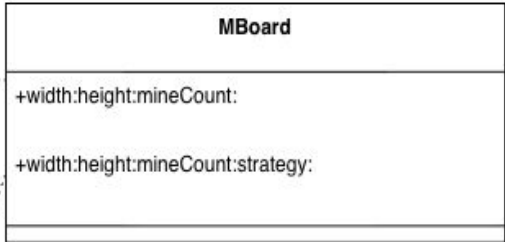
Design après ce challenge :

METHODE 1 : LE SELECTEUR ALEATOIRE

width: w height: h mineCount: m

1. strategies := { Random, LowerHalf, OddCols }
2. s := strategies atRandom
3. ^ self width: w ... strategy: s

(Il choisit une stratégie et appelle la méthode 2)



Utilise (Délégation)

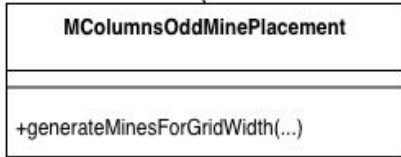
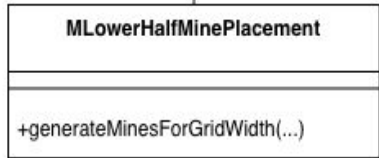
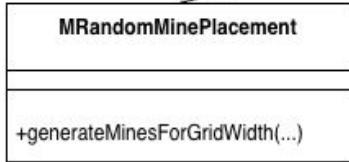


METHODE 2 : LE CONSTRUCTEUR DELEGUE

width: w height: h ... strategy: s

1. coords := s generateMinesForGridWidth: w ...
2. Save last params (Replay)
3. Build Matrix using coords

(Il ne calcule rien, il utilise 's')



Éléments Importants

- Design pattern Strategy => Meilleur flexibilité.
- Défi : Contrainte géométrique vs Nombre de Mines :
 1. **Problème** : Les stratégies (ex: LowerHalf) réduisent l'espace disponible.
 2. **Risque** : Si le joueur demande plus de mines que d'espace libre -> Crash ou Boucle infinie
 3. **Solution** : Le Fallback Automatique
 1. Détection du manque de place.
 2. Bascule dynamique vers MRandomMinePlacement.

CHALLENGE N°4 : POSSIBILITÉ DE REJOUER LA MÊME ANCIENNE PARTIE

Play

Replay last Game ►

GAME 1

0	0	0	0	0
0	1	2	2	1
0	1			3
0	1	4		
0	0	2		3

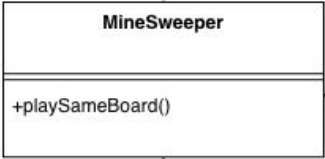
REPLAY OF GAME 1

0	0	0	0	0
0	1	2	2	1
0	1			3
0	1	4		
0	0	2		3

Design après ce challenge :

MineSweeper >> playSameBoard

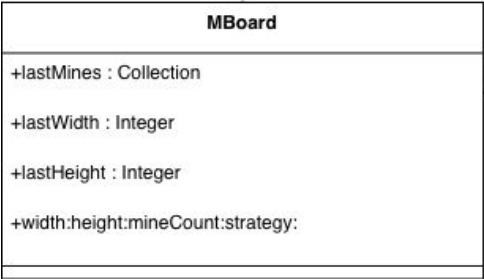
strategy := MReplayMinePlacement new.
strategy minesToReplay: MBoard lastMines.
MBoard ... strategy: strategy.



MBoard (Class Side)

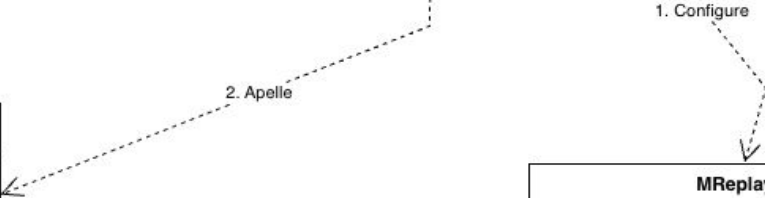
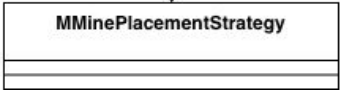
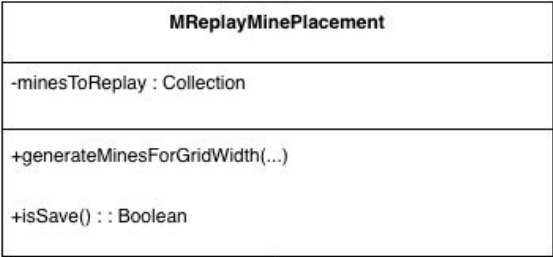
Stockage statique :
- lastMines
- lastWidth

Sauvegarde automatique
si la stratégie n'est pas un Replay.



MReplayMinePlacement

- minesToReplay: (Injecté depuis MBoard)
- generateMines: Renvoie directement la liste
- isSave: ^ false (On ne resauvegarde pas un replay)





Merci de votre attention

Avez-vous des questions ?